

Governed Autonomous Execution: Authority Separation and Semantic Invariance Across Execution Boundaries

A. Figueroa, Crystal Warden Labs

Preprint, version 1.1, October 2026. doi:10.5281/zenodo.23073693

Contents

Abstract	2
1. Introduction	2
2. What governed autonomous execution means	3
3. Primitives	4
4. The architecture: two pillars and the relevance boundary	6
5. The distinctions, and their two roots	7
6. The formal model (machine-checked)	9
6.1 Authority and its equivalence	10
6.2 Admissibility and non-expansion	10
6.3 Relevance, stated on its own terms	11
6.4 The closure hazard	12
7. The originating corpus, and what each level of evidence establishes	13
7.1 The machine-checked bridge	13
7.2 Node-level governance, implemented and tested	14
7.3 Distributed governance, implemented and tested	15
8. Evidence: an external countermodel search	16
9. Related work	18
10. Limits and open problems	19
11. Conclusion	21
AI use disclosure	21
References	21
Appendix A. The dictionary reduction	23
Appendix B. The formal development	24
Appendix C. External mappings, with grades	25
Supplementary artifacts	27

Abstract

Autonomous systems are governed in pieces, with different bodies of work constraining the actor, the proposed action, the API request, the runtime trace, the execution environment, or the effect, and each making a different object primary. We ask whether governed autonomous execution can instead be organized around a single question, what a system is permitted to do here, answered by an authority relation that does not require a model of the worker that produced the proposal or of the substrate on which it runs. The model that results separates the properties that must not be mistaken for authority from the differences in representation that may safely be discarded because they cannot change what is authorized, and it is deliberately small, built from a handful of primitives and organized by two pillars we call authority separation and semantic invariance. We give a machine-checked formal core, proved in Lean without dependence on any implementation, in which the formalization also exposes a design condition for implementations that group states by a pairwise relevance criterion and take its transitive closure, shown with a concrete counterexample, and we machine-check that Figueroa quantization, a transformation from the system the theory was reduced from, is a sound relevance criterion whose equal quantizations preserve the full routing result over the policy's authored action domain, instantiating the model's admissibility and non-expansion results. We then run an external search for counterexamples and report graded evidence that ranges from direct instantiations in independently built systems down through structural analogues to failures we can only interpret through the theory. Generality, throughout, means the theory requires no model of the worker or the substrate, not that it is proven complete over all autonomous systems.

1. Introduction

Governing what an autonomous system does is not a new problem, and it is not solved in one place. Existing work governs different parts of execution, constraining the actor by shaping or filtering what a model may produce, checking a proposed action before it runs, or governing the API request, the runtime trace, the execution environment, or the effect after the fact, and each of these is reasonable for the part it addresses. Several contemporary systems already reason at the execution boundary rather than only at the actor, with runtime shields keeping an agent to safe actions, runtime rule engines enforcing constraints before an unsafe action runs, and proof-of-execution work making an effectful execution the object of verification under a contract. We are not claiming that this work governs the actor and misses the act, but asking a different question, whether the governance of a consequential act can be described by one authorization abstraction that does not require the worker itself to be represented as a theoretical primitive, so that the theory models the authority over the act rather than the worker or the substrate.

We propose governing the act rather than requiring the theory to model the actor. That is an abstraction choice, and it is the source of whatever generality the

theory has, because a worker may be a language model, a human, a controller, a program, or an organization the theory never describes. The paper makes four contributions. First, a proposed conceptual model reduced from an existing implementation vocabulary, through counterexample and explicit definitional choice, with five primitives, two pillars, and a family of distinctions. Second, machine-checked formal consequences of the model, including one design condition the definitions alone did not reveal. Third, an external countermodel search with graded mappings, testing whether those distinctions recur outside the system they came from. Fourth, a first-party implementation corpus with a formally checked bridge from the model to one built transformation.

One deliberately mundane example runs through the paper, so the ideas attach to a concrete case before they are generalized. A controller governs a valve. A pressure sensor reports 40, the controller proposes OPEN, the policy permits OPEN while pressure is below 50, the authority relation returns yes, the valve opens, and a signed record keeps enough to reconstruct why. Nothing here is about AI, and the same structure appears whether the worker is a controller, a language model, or a person. We return to the valve as each idea is introduced.

2. What governed autonomous execution means

Return to the valve. The controller proposes OPEN, the policy permits OPEN while the pressure stays below 50, the authority relation returns yes for the reading of 40, the valve opens, and a signed record keeps enough that the decision can be reconstructed afterward. Governed autonomous execution, as we use the term, is what that scene has in common with any other governed action, and it has three parts, a consequential action that will have an effect once it happens, a bounded permission that independently determines whether the proposed action may happen now rather than treating proposal itself as authorization, and a verifiable governing under which the claim that the system governed the action can be checked afterward from a record rather than taken on trust.

The third part, verifiability, is a deliberate choice. Consider a hardware interlock that permits only safe transitions and keeps no record at all. It governs, in a weaker and perfectly reasonable sense, and in that sense no record is required. We require the stronger sense because we want the claim of governance to be testable from outside the system, and without a record a governed system cannot be told apart from an ungoverned one that merely happened to behave. This choice is what makes evidence a primitive in section 3.

The generality claimed here is narrower than it sounds. It means the theory does not require a model of the worker that produces a proposal or of the substrate that runs it. It does not mean the worker never matters, since a policy that lets Alice open the valve and not Bob depends on who is asking, and the model carries that through authenticated identity, role, delegation, or enrollment in the authority-relevant state, where the first root of section 5 lets such a property condition a grant without conferring authority on its own. Nor does it mean

the primitives are proved minimal or complete over all autonomous systems.

3. Primitives

We did not begin with a vocabulary designed for a theory. We began with the originating implementation's working dictionary of about one hundred and twenty terms and classified its entries as candidate primitives, derived concepts, general mechanisms, implementation details, or evidence records. That exercise exposed recurring roles beneath the implementation's specific names. The proposed model retains five primitives, **STATE**, **PROPOSAL**, **AUTHORITY**, **ACTION**, and **EVIDENCE**. We also name three terms needed to explain how the model is applied, **DECISION**, **EXECUTION BOUNDARY**, and **WORKER**, without treating them as additional primitives. The reduction is an explanatory compression of one originating system, not a proof that these five primitives are uniquely minimal or complete.

We present each of the five alongside the attempted elimination or the definitional choice that led us to retain it.

STATE is what is known at the governed boundary. The attempt to eliminate it observes that an unconditional authority, a rule that says never execute a given action, needs no observed state at all. State survives this as the empty state. Authority over the empty state is a constant function, and the rule that ignores the world is the special case, not a refutation. State is therefore kept, with its definition widened to admit the trivial state.

PROPOSAL is a candidate consequential transition offered for authorization. The attempt to eliminate it observes that a system can act on a fixed schedule with no proposer in sight, a watchdog that fires every second. Proposal survives this because the scheduled transition is itself the proposal. A proposer need not be a worker, since a clock, a rule, or the environment can propose. Proposal is kept, with its definition widened past worker-emitted candidates.

AUTHORITY is the relation that determines whether a proposed transition is permitted under a given state. State, proposal, action, and evidence can describe what was considered, what occurred, and what was recorded, but none of them determines whether the action was permitted. That relation is what we call authority, and it does not derive from the other four, since they together do not yield permission. Permission is the added relation, and this reduction could not eliminate it without reintroducing permission somewhere else.

ACTION is the consequential transition whose execution is at stake. Without it, a model can represent state, candidate proposals, permission, and evidence, and still not name what permission ranges over, the consequential transition that authorization is authorization of. Action is that transition. It is distinct from proposal, which is only a candidate, and from decision, which is only the ruling on a candidate.

EVIDENCE is verifiable information sufficient, alone or together with referenced artifacts, to re-derive a decision. It is the one primitive that rests on the definitional choice of section 2: under verifiable governance a system that keeps no record cannot demonstrate it governed, so evidence is required, whereas under the weaker record-free notion it would not be primitive at all. The record-free interlock of section 2 is the case that choice excludes.

Three further terms are not primitives.

DECISION is derived. A decision is authority applied to a proposal under a state. We name it because it recurs throughout, not because it is a separate object.

EXECUTION BOUNDARY is a defined locus, not a sixth primitive. It is the place where an authorized action becomes consequence. Two systems with the same state, proposal, authority, action, and evidence do not differ in theory because one terminates at an indicator light and the other at a network packet. The boundary is a relation of action, not an object on the same level as the five.

WORKER is external by design. The actor that produces a candidate is untrusted and is not itself modeled, although authority-relevant facts about it, such as an authenticated identity, a role, or a delegation, may be components of state, and keeping the actor outside the primitive set is what it means to govern the act rather than model the actor. A worker may be a language model, a human, a controller, a program, or an organization the theory never describes.

PROPOSAL and ACTION deserve a defense as separate objects, since a natural reduction would collapse them into one TRANSITION carrying a proposed and an executed predicate. We keep them apart because the distinction is operational, not terminological. PROPOSAL denotes the candidate transition presented to authority, and ACTION denotes the consequential transition whose occurrence is governed. Collapsing them removes the ability to represent disagreement or non-correspondence between what was proposed and what occurred: a candidate refused and never executed, an authorized proposal that fails before execution, an executed action that differs from what was proposed, and evidence that independently binds proposal, authorization, and execution. The public implementation record shows the distinction operating at more than one level. Nodes each report a reading and a collective fuses them into one posture under a signed table (ledger #101). A receiver's admission policy authorizes a transported decision, refuses it with a named cause, or abstains when it lacks the state the policy needs (ledger #150). An actuator on another node moves only on an admitted action signed by the authority, refuses replayed, spliced, unsigned, and refused-admission actions, and answers with receipts that bind the decision, the action, and the outcome (ledger #157). In each, a candidate can be refused or abstained on rather than executed, and a separate record binds proposal, authorization, and execution. This makes the defense testable. A reduction to four objects succeeds if it recovers these distinctions and the results below without reintroducing the proposal-action difference as a subtype

or a state, and we have not found one.

We arrived at these five objects through reduction of a larger implementation vocabulary, counterexample construction, and explicit definitional choices, and the formal development in section 6 requires no further primitive. Some of the five are choices as much as findings, and we say which. `AUTHORITY` and `ACTION` resist elimination on structural grounds. `STATE` survives by admitting the empty state, `PROPOSAL` by broadening the proposer past a worker, and `EVIDENCE` only because section 2 chose verifiable governance as the noun.

4. The architecture: two pillars and the relevance boundary

Start from the valve again. The policy reads the pressure against its threshold, which makes the pressure a fact the decision depends on, while the color of the wire that carried the reading is not, and could be recoded or dropped without changing what the valve is permitted to do. Every governed system faces that same split between the differences that could change what it is permitted to do, which we call decision-relevant, and the differences that could not and may safely be discarded, and the architecture of the theory is the machinery that keeps the two apart. It comes down to two questions that recur through the rest of the paper, what must stay different and what may safely become the same, and the two answers are the pillars the section is named for. Authority separation is the answer to the first and semantic invariance to the second, and over both stands one organizing principle rather than a law, that governed execution preserves every distinction necessary to determine authority and permits every transformation that preserves that determination. The rest of the section develops each pillar in turn and then the relevance rule that decides, for any given difference, which pillar it falls under.

The first pillar is authority separation. It protects the distinctions whose collapse would create, expand, or obscure authority. A property that travels alongside authority is not authority, and a system that reads it as authority has granted permission it never held.

The second pillar is semantic invariance. It permits the transformations that leave the authorization semantics unchanged. A decision may move across representations and substrates, and as long as what is permitted does not change, the movement is admissible.

The rule that decides which pillar a given difference falls under is the relevance principle, already stated informally as preserve the decision-relevant differences and discard the rest, and it is the seam where the two pillars meet, with an exact form.

The valve makes that form concrete. Suppose the policy permits `OPEN` whenever the pressure is below 50, so that a reading of 40 and a reading of 41 are

different measurements but equivalent under this authority, because it returns the same answer on both, while a reading of 51 cannot join them because it changes the answer. Generalizing, authority induces an equivalence relation on states, under which two states are equivalent when they authorize every action identically, that is when for every action the authority returns the same outcome on both. Inside an equivalence class the distinctions may collapse and semantic invariance holds, while across classes exact preservation no longer holds and authority separation bites, so the two pillars are the inside and the outside of one relation induced by authority. A transformation that stays within an authority-equivalence class preserves the authorization exactly, and one that crosses classes must instead pass the weaker safety test of non-expansion developed in section 6. The formal treatment there defines this relation, proves it is an equivalence relation, and builds the rest of the safety results on it, and the constructive example is a quantization that collapses state values precisely where the authorization semantics are unchanged.

The equivalence construction itself is elementary. Quotienting by the fibers of a function preserves that function. The substantive proposal is not that every function induces equivalence classes, but that authority is the relation around which governed execution should preserve and collapse distinctions, and that the eight distinctions developed in section 5 identify where that choice has operational consequences.

The equivalence by authority has a close analogue in work on action-sufficient compression, where the coarsest compression that preserves the optimal action is the quotient of a support space by policy equivalence, merging two states exactly when they require the same action (Walsh 2026, action-sufficient compression). That is the same move as our quotient of state by authority-equivalence. The question differs: that line asks what must survive compression to preserve action adequacy, and this paper asks what distinctions must survive transformation to preserve authority over a consequential action, adding authority separation, non-expansion, and the evidence and boundary structure around it.

5. The distinctions, and their two roots

On the valve the controller is physically wired to open it, it asked to open it by proposing OPEN, its request was validly signed, and the scheduler happened to run it next, and yet not one of those four facts is the policy saying yes. Each of those properties travels alongside authority without being authority, and reading one of them, being able to act, having proposed, having signed, or being next to run, as being allowed is the failure behind the first of the two roots.

There are eight such distinctions in all, operational distinctions that recur wherever autonomous execution is governed and whose collapse we have observed, in the implementation and in outside systems, to expand or obscure authority. Six of them instantiate two generating roots, and we reserve the word separation

for those six, calling them proposed separations rather than axioms because they reduce to the roots rather than standing as independent foundations. The remaining two belong to other parts of the model, one to the relevance principle and one to the evidence primitive.

One point of method is worth settling before the roots. The separations remain distinct failure modes even where they share a root, so reducibility here concerns explanation rather than identity, and saying that two separations share a root is neither saying they are the same distinction nor claiming that each can be collapsed in isolation, which we do not establish.

The first root states authority separation over an arbitrary property, that no property which correlates with authority confers authority except through the authority relation itself. Without that last clause the claim would be false, because a policy is free to make a property a condition of a grant, so an authority whose rule is to permit exactly when some property holds is admissible, the property there conferring authority through the relation rather than on its own. What the root forbids is a property acquiring authorization force by merely holding, outside the relation, and four of the six separations are this one root applied to the four properties most often mistaken for authority, namely capability, the ability to cause an effect, then proposal, the selection of a candidate action, then authenticity, a validated identity or signature, and finally sequencing, the power to determine what runs next, what is visible, or what is reachable. The four collapses therefore share one root, a property read as authority, applied to four different properties.

The second recurring problem is confusing a representation with what the decision actually means, and that is the second root, the representation principle, that a representation is not its referent and that only the decision-relevant distinctions in a representation are authority-bearing. The other two separations are this root on two planes. On the state plane, a representation of state is not the decision semantics, and a check performed on one representation must not be bypassed by another representation of the same meaning. On the effect plane, a record of an event is not the event, and the evidence that an action occurred is a distinct object from the action, distinct enough that either can exist without the other. On the valve, 40 psi and an equivalent re-encoding of the same reading must authorize alike, and a record saying the valve opened is not the valve opening.

The remaining two distinctions do not reduce to a root, and we place them precisely rather than force them.

One is the distinction between absence and explicit state, where a missing value is not a value and absence must not silently become an ordinary state that changes the decision, so that on the valve a missing pressure reading must not quietly become 0, which would read as below 50 and authorize OPEN. This sits under the relevance principle as a boundary condition, so that wherever absence can alter authorization it must remain distinguishable from an explicit value,

which is what stops an implementation from assuming that missing means false, zero, or normal when the policy distinguishes them.

The other is the distinction between provenance and truth. Authenticating a representation establishes who asserted it and how, and it does not establish that what the representation asserts is true. On the valve, a signature proves which sensor reported 40, not that the pressure was 40. This is not a property mistaken for authority, and it is not the bare representation root either, because it is specifically the limit of what evidence can certify. We therefore place it as the defining semantics of the evidence primitive. Evidence can certify the governed path, its provenance, and the recorded decisions, to the extent its verification mechanism supports, and it cannot thereby certify the truth of the world the readings purport to describe. It is therefore one of the eight distinctions but not one of the six separations.

Time is not a ninth distinction. Versioning, revocation, a monotonic floor, and a stale witness all enter through the authority relation itself, evaluated against the state in force. We write authority as a relation over a full authority-relevant state that carries whatever the system's authority depends on, including the decision time, the policy version in force, and its validity. Version and validity are then movements within that state, not a new object. Later information does not alter the authority relation that was evaluated at decision time, although it may alter what a later observer can establish about the validity or trustworthiness of that evaluation. If evidence later shows a key was already compromised at decision time, a system may legitimately conclude the supposed authorization was never trustworthy, though what it cannot do is change which relation was applied then. The theory governs authorization at decision time and preserves the record.

Two roots, authority separation over any authorization-correlated property and the representation principle, generate the six separations. Of the remaining two distinctions, absence versus explicit state is a boundary condition of the relevance principle, and provenance versus truth is the semantic limit of the evidence primitive.

6. The formal model (machine-checked)

The safety content of sections 4 and 5 is small enough to check by machine, and we formalized it in Lean 4. The formal model imports neither a mathematics library nor the implementation corpus, and it is parameterized over the outcome type, so nothing in it is special to a two-valued permit and deny. We give the logical development here and put the Lean names in parentheses only where they aid traceability, and the full index of definitions and theorems is Appendix B. The formalization also exposed one design condition, the closure hazard of section 6.4. As to scope, the machine-checked core covers the semantic-invariance pillar and the relevance seam over an abstract authority relation. The authority-separation pillar of section 5, the EVIDENCE primitive, and the execution boundary are argued in prose and are not part of the Lean. The core

also abstracts the distinction between PROPOSAL and consequential ACTION at the authorization step, representing the candidate presented to authority as the action argument, so it proves properties of authorization semantics and not correspondence between a proposal, a ruling, and a subsequently executed consequence, which remains part of the conceptual model and the implementation evidence. The safety results below should therefore not be read as an end-to-end proof of governed execution.

6.1 Authority and its equivalence

Model authority as a function A from a state and a candidate action to an outcome, written $A(\sigma, a)$. The outcome is drawn from a type carrying a no-more-permissive-than preorder, written $x \preceq y$. A permit-only two-valued type is one instance, and the five-valued codomain used later, refuse, deny, park, abstain, authorize, with authorize its top, is another. Authority induces an equivalence on states,

$$\sigma_1 \sim_A \sigma_2 \quad \text{exactly when} \quad \forall a, A(\sigma_1, a) = A(\sigma_2, a).$$

Two states are equivalent when they authorize every action identically (in Lean, `simA`, shown reflexive, symmetric, and transitive by `simA_equiv`). This relation is the formal seam of section 4. Inside a class, distinctions may collapse under exact preservation. Across classes, exact preservation fails, and a transformation must instead satisfy the weaker non-expansion condition developed next.

6.2 Admissibility and non-expansion

A transformation T of states is admissible over a domain when it preserves the committed outcome for every action on every state in the domain, $A(T(\sigma), a) = A(\sigma, a)$. Admissibility is semantic invariance made checkable, since T may change the representation of the state as long as the authorization is unchanged (in Lean, `Admissible`, with `admissible_preserves` showing that an admissible T lands each state back in its own $\sim_A$ class).

Admissibility is stronger than safety needs. Sometimes a system cannot establish enough to preserve the previous decision and has to fall back, and that must be allowed. In the originating implementation, a node that stops hearing a peer reads it as an explicit STALE input that the signed table decides on (ledger #101, #136), a receiver that lacks the state a decision needs abstains (ledger #150), and a consumer that loses its sender's stream parks on its signed fail-safe once the stale bound passes (ledger #125), in each case rather than repeating its last decision. That deliberately does not preserve the previous outcome, but it also grants no new authority. The weaker guarantee that captures it is non-expansion: a transformation is non-expanding when it leaves every outcome no more permissive than before. Formally, $A(T(\sigma), a) \preceq A(\sigma, a)$ (in Lean, `NonExpanding`). Generally that is all it asserts, that the outcome does not

move up the permissiveness preorder. For a codomain in which authorization is the unique permissive top, including the Verdict instance below, non-expansion entails that a transformation cannot turn a non-authorizing outcome into an authorization, the informal reading that nothing becomes authorized that was not authorized before, while over a general preorder that reading is instance-specific and the general claim is only the order inequality.

Admissibility and non-expansion relate in one direction only. Every admissible transformation is non-expanding (in Lean, `admissible_nonexpanding`), because equality is a special case of the order, and in the Lean exact preservation is the admissibility predicate itself, so there is a single nontrivial step. The converse fails, since a transformation can lower an authorization it cannot establish, which is non-expanding but not admissible (in Lean, `nonexp_not_admissible`). Admissibility preserves autonomy as well as safety, and non-expansion preserves only safety. The stale node's abstention is a contraction that non-expansion permits and admissibility forbids.

6.3 Relevance, stated on its own terms

A relevance analysis is what a governor uses to decide which state distinctions may be collapsed. It is tempting to define a safe analysis as one that yields a non-expanding transformation, but that is circular, because safety would then be its own definition. We state the criterion independently instead. A relevance criterion R is a relation on states, and it is sound when every pair it declares interchangeable is authority-equal, so that $R(\sigma_1, \sigma_2)$ implies $A(\sigma_1, a) = A(\sigma_2, a)$ for every a (in Lean, `Sound`). Soundness mentions neither admissibility nor non-expansion. A sound criterion whose induced transformation only ever maps a state to one the criterion already declares interchangeable with it is admissible, and therefore safe (in Lean, `sound_induces_admissible`). Safety now follows from a property of the criterion rather than being assumed of the transformation.

Completeness is the converse. A criterion is complete when it declares interchangeable every pair that actually is authority-equal (in Lean, `Complete`), and a criterion that is both sound and complete is exactly authority-equality (in Lean, `sound_complete_char`). Soundness is the no-unsafe-merge half. Completeness is the no-missed-equivalence half. Under the analysis construction below, missed equivalences can appear operationally as unnecessary abstention.

The gap shows up at the level of the analysis a governor actually runs, which is a separate object from the criterion. An analysis abstains when, faced with a state it cannot place, it emits a conservative floor rather than the true authority. A false abstention is emitting that floor where the true authority was not the floor, which is unnecessary loss of autonomy relative to the modeled authority (in Lean, `FalseAbstain`). Two levels must be kept apart here. At the analysis level, an analysis that commits to the true authority everywhere has no false abstention (in Lean, `complete_no_false_abstain`, hypothesised on analysis-completeness), while a merely sound analysis can still abstain falsely (in Lean,

`sound_allows_false_abstain`), which is the permitted asymmetry, autonomy may be lost while authority is never expanded. The bridge from the criterion level is that a sound criterion whose induced transform places every state yields such a complete analysis (in Lean, `sound_induces_complete_analysis`), so criterion soundness plus total placement, not criterion completeness by itself, is what rules out false abstention.

The rate at which that happens can be measured. Over an explicitly finite domain D we count the false abstentions (in Lean, `faCount`), and the rate is that count over the size of D and is defined outside Lean, while the machine-checked object is the Nat count, which assumes decidable equality on outcomes. If every situation in which one analysis falsely abstains is also one in which another does, with the underlying authority and the floor fixed, the first has the smaller or equal count (in Lean, `faCount_mono`). A sound analysis that places every state has a zero count (in Lean, `sound_total_faCount_zero`), so by contraposition, for a sound analysis a nonzero count reflects states the criterion could not place, not irreducible uncertainty. A zero count over a sampled subset of D does not establish completeness over all of D . The count is a property of the domain it is taken over.

6.4 The closure hazard

Pairwise agreement does not make a safe group. State one may be similar enough to state two, and state two similar enough to state three, without state one being similar enough to state three. If an implementation turns those pairwise matches into one bucket, by taking their transitive closure, it can erase a distinction that changes authorization. Whether that happens depends entirely on what the local match means.

Stated formally, a criterion is a relation, and forming buckets takes its transitive closure, and the question is whether soundness survives. For an exact criterion it does: if every declared pair is authority-equal, the closure relates only authority-equal states, because authority-equality is transitive (in Lean, `sound_tc`), so bucketing under an exact criterion is safe. For a criterion that is merely similar enough it does not. If the local test is that two states agree on some action, a plausible and too-weak heuristic, then states one and two may be declared interchangeable on one action, two and three on another, while one and three agree on no action, and the closure merges one and three and collapses a distinction that changes authorization (in Lean, `weak_criterion_not_closed`, a concrete three-state, two-action witness). The design condition this produces was not visible from the definitions. Pairwise overlap is insufficient for safe bucketing. A bucketing criterion must be strong enough that its transitive closure remains within authority-equivalence, and full authority-equality is one criterion that satisfies that condition, and a relation finer than authority-equivalence could also serve. The underlying fact about transitive closures of non-transitive relations is elementary.

7. The originating corpus, and what each level of evidence establishes

The formal model stands on its own. Nothing in section 6 refers to the implementation, the general file imports neither a mathematics library nor the corpus, and only the bridge file imports it, so the dependency runs one way.

The theory was also reduced from that implementation. The primitives and the distinctions were found by building a policy engine, extending it across execution environments and distributed flows, and meeting the questions of authority, abstention, evidence, freshness, and collective decision through the implementation rather than ahead of it. The model's definitions do not depend on this history, and the history is not evidence that the model is general. A single originating corpus cannot establish generality, and section 8 tests whether the same distinctions recur in independently built systems.

The corpus supports several kinds of claim at several strengths, and we keep them apart rather than letting the strongest color the rest. A formal development establishes a property under stated hypotheses. Implemented machinery establishes what the system is built to express and evaluate. Tested behavior establishes what was observed under the conditions exercised. Implemented and tested claims cite rows of the PrismPath evidence ledger, a public, append-only record in the same repository whose revisions are anchored with OpenTimes-tamps, written ledger #N. The table maps the theory's principal elements to representative artifacts, the kind of evidence each supplies, and its limit.

Theory element	Representative artifact	Evidence	Limit
Relevance, which distinctions may collapse	<code>decision_preservation</code> , <code>fq_sound</code> (ledger #140)	machine-checked	under the Level M hypotheses
Authority separation, proposal apart from authorization	authored routing apart from framework verdicts	implemented	not shown universal
Authored routing outcomes, abstain, escalate, refuse, deny	<code>receiver_admission.md</code> , <code>ai_action_gate.md</code> , <code>overlay.md</code> (ledger #150)	implemented, tested	one corpus, authored scenarios
Evidence, provenance apart from truth	signed receipts with a cause code (ledger #157)	implemented	external anchors remain
Reuse of the governing shape one level up	<code>mesh-fusion/</code> , <code>room_fusion.md</code> (ledger #101, #136, #151)	implemented, tested	no composition theorem
Conservative fallback	<code>route:stuck</code> at a node, a STALE input the signed table decides on in a collective (ledger #101, #151)	implemented, tested	on the inputs exercised

7.1 The machine-checked bridge

Figuroa quantization quantizes a reading before the policy routes it, and the corpus carries a proved theorem, `decision_preservation`, that two well typed readings with the same quantization route identically. The bridge connects that

theorem to the general model over the policy's actual action domain rather than a single dummy action, and it is the one file in the formal development that imports the corpus.

Three statements carry it, in order of what they establish. `fq_route_eq` says equal quantizations of well typed readings produce the same routing result, the whole `Option Action` a rule selects, taken straight from `decision_preservation`. `fq_multi_action_equiv` lifts this to authority, so the two readings carry the same authorization outcome for every action in the policy's domain rather than one selected action, which is the state equivalence of section 6.1 read over the routing authority. `fq_sound` states that Figueroa quantization is a sound relevance criterion in the sense of section 6.3, so it is a concrete realization of the relevance principle, a rule for which distinctions in a reading may be discarded without changing what the policy authorizes. On these rest `fq_admissible`, which shows that any transformation preserving quantization and well typedness on a domain preserves the per action authority and so is admissible, and `fq_nonexpanding`, which derives non-expansion from admissibility through the general safety arrow on the Boolean per action codomain.

We are exact about the reach of this. It covers deterministic Level M routing over the authored action domain. Authored outcomes are the route targets, so an abstain, an escalate, a refuse, or a deny that a policy names is an ordinary action in this domain and is preserved exactly as an affirmative action is. The implementation type `Action` here names an authored routing target, including governed non-action outcomes, and should not be confused with the theory's `ACTION` primitive, which names the consequential transition whose execution is at stake. The bridge proves preservation of the routing decision, not correspondence between that decision and a subsequent physical effect. The value `none` means that no authored deterministic route matched. When no semantic edge is available, the engine reports this fall-through as `route:stuck`, cause 36. Neither `none` nor that framework generated stop is an authored refusal. What the bridge does not touch is the framework verdicts the runtime emits outside the authored node set and the semantic routing path, both of which appear at the next two levels as implemented and tested behavior rather than as proof.

7.2 Node-level governance, implemented and tested

A node policy is a graph of authored nodes and routing edges, and its outcomes are the nodes the walk can reach, so a governed non-action is authored the same way an action is. The hardware admission policy `receiver_admission.md` routes to `refuse_window`, `refuse_replay`, `refuse_policy`, `refuse_chain`, `refuse_route`, and `abstain` under explicit conditions and to `authorized` otherwise, the action gate `ai_action_gate.md` routes to an authored `abstain` when worker confidence falls below its floor, and the operator overlay routes among `escalate`, `deny`, `allow`, and `observe`. In each the same evaluator governs affirmative action and governed non-action.

The runtime also emits a fixed vocabulary of conservative verdicts outside the authored node set, and we keep these separate from authored outcomes. The engine stops with `needs_human` when a worker requests a human or when semantic confidence falls below the calibrated floor, with `route:stuck` when a non total node has no viable edge, with `max_steps` when the signed step bound is spent, and with `contract_violation` when worker output breaks the declared contract, each carried on the receipt as a stopped state and a cause code, a codomain distinct from the action a policy routes to. This behavior is implemented and exercised by the conformance harness. It is first party and over authored scenarios, so it establishes what the system does under those conditions and not a general property.

7.3 Distributed governance, implemented and tested

The same propose then decide shape appears one level up, which changes the level at which the candidate decision is produced without changing the governing idea. In the governed ESP-NOW mesh, three nodes each sense one channel and broadcast a band, and every node evaluates the same signed Level M table over all three to reach one fused posture, computing it identically at every tick (ledger #101). The collective changes its own policy under the same discipline, since a pushed table is verified on every follower against the authority's signature and a monotonic version floor and commits only on a quorum of acknowledgments, and a change without a live quorum is refused (ledger #100, #102, #136). On the camera relay, two cameras are fused into one room verdict under a policy of its own (ledger #151), and the same table image decides that verdict in a Linux kernel, 47 of 47 ticks agreeing with the relay (ledger #153).

In the collective the conservative fallback is authored rather than emitted by the engine. A node unheard past its freshness window becomes an explicit STALE input that the signed table decides on, so the fleet keeps deciding rather than going silent (ledger #101), a node partitioned on real radios escalates the fused verdict while the surviving quorum keeps deciding and is re-absorbed on return (ledger #136), and a camera that goes quiet reads as a STALE state the room policy decides on (ledger #151). This is the distinction between absence and explicit state of section 5 at the level of the collective. These behaviors are implemented and tested on the benches and field walks the ledger records.

The reuse across levels is an architectural claim supported by tests, with the machine-checked bridge as its single formal anchor. We do not prove a composition theorem, and we do not read universal non-expansion off the scenario tests. Across the three levels, one decision preserving, policy governed approach is instantiated at a node, across execution environments, and across a collective, and at each level lost state becomes an explicit input or outcome the policy governs rather than silence, with formal proof at the routing core and implemented and tested evidence around it.

8. Evidence: an external countermodel search

The distinctions of section 5 were extracted from one implementation, and that is a weakness until they are tested elsewhere. If they are artifacts of that implementation, systems built independently should not systematically preserve them. So we searched for countermodels. For each distinction we asked whether an unrelated, mature system keeps it, and whether collapsing it there is a recognized failure. We report what we found, and we grade the evidence, because the grades are not interchangeable. A direct instantiation is a system that keeps both sides of the distinction apart as a stated architectural boundary. A structural analogue exhibits the same shape but does not instantiate the theory exactly, or rests partly on our reading. A theory interpretation is a known failure that the theory explains as a collapse, where the external literature does not itself use our terms.

Six of the eight distinctions are direct instantiations, a grouping by evidence that cuts across the grouping by root in section 5, each with a canonical source, and a source that only resembles a distinction is not counted as a direct instantiation.

- Capability versus authority. The ability to cause an effect is not permission to cause it. We mean capability causally, not a capability-security token that embodies delegated authority. The purest causal instance is Unix ambient authority, where a setuid or capability-bearing process can perform a syscall it is not permitted to perform on a caller's behalf, and the deployed named boundary is mandatory access control over discretionary access control, where a separate authorization decision is applied on top of whatever a subject can otherwise do (Bell and LaPadula 1973; Loscocco and Smalley 2001).
- Proposal versus authorization. A request is not a grant, kept in access-control architectures as the split between the point that receives a request and the point that decides it (OASIS XACML 3.0).
- Authenticity versus authority. Who is speaking is not what they may do, and authentication is not authorization, with token scope and audience checked apart from identity (RFC 6749; OpenID Connect Core 1.0; RFC 8707).
- Absence versus explicit state. Missing is not present. Fail-closed design keeps the deny direction (Saltzer and Schroeder 1975), and SQL three-valued logic keeps a null distinct from an ordinary value rather than coercing it (Codd 1979).
- Representation versus semantics. A form is not its meaning, and the standing practice is to canonicalize before the decision that reads it, most pointedly a path before an access-control check, so the decision is taken on the meaning and not an alternate encoding (Unicode UTS 39; OWASP ASVS 5.0.0).
- Provenance versus truth. A record certifies its own provenance, not the world, and supply-chain attestation scopes itself to provenance integrity and explicitly not to the soundness of what was built (SLSA specification;

Torres-Arias et al 2019).

Two of the eight are structural analogues, marked as such rather than over-claimed.

- Sequencing versus authority. Orchestration systems separate scheduling from admission: a mature platform runs authentication, then authorization, then admission control before it persists the object, with scheduling a separate later control loop (Kubernetes documentation). This supports the broad observation that ordering and permission are separated, but does not by itself establish that sequencing can expand authority in general, so we keep it structural pending a tighter source.
- Execution versus evidence. An effect can diverge from its record, most cleanly in the dual-write problem (Richardson 2018; Waldron 2024). The shape is right, but the external record is weaker than the theory's evidence, a verifiable object from which the decision is re-derivable, so this is an analogue rather than an instantiation.

One mapping is a theory interpretation. HTTP request smuggling is a known failure in which two intermediaries disagree about how to frame the same bytes (Linhart et al 2005; PortSwigger). The theory reads it as a disagreement over whether a representation transformation is admissible, two evaluators returning different decisions on one input. The external literature does not describe it in those terms, so we classify it as our interpretation rather than an independent instantiation.

In the systems surveyed, we did not identify a clear case in which one of the proposed distinctions was collapsed while the corresponding governance property remained intact. That does not show that no governed system can collapse them. What the search establishes is recurrence, meaning that distinctions extracted from the originating implementation also occur as explicit architectural boundaries in independently developed systems. It is not evidence that the distinctions are universally necessary.

One derivation ran in the other direction. Applying R1 to urgency yields urgency is not authority, a candidate separation we wrote down before searching for a system. A subsequent search identified break-glass access controls as an existing realization of that distinction (Brucker and Petritsch 2009), which grant urgent access without letting urgency itself become authority, routing it instead through a separately governed, bounded, and logged path to exceptional authorization. Our working notes record the urgency candidate before the external search, but those notes are first-party and not independently timestamped, so we report the ordering rather than offer it as evidence, and in any case it is derivation before lookup, not an independent prospective prediction. Because R1 yields a candidate separation for every authorization-correlated property, finding one with an external realization is weak evidence.

9. Related work

Governing what a system does at the point of action is not new, and the contribution here is not an unoccupied technical layer. Several bodies of work govern autonomous execution, and the clearest way to place this theory among them is by the question each asks and the object each makes primary. The lineage is longer than a recent convergence, since runtime enforcement of safety around an autonomous agent predates the current interest by years.

Shielding asks whether a candidate action satisfies a safety specification before it reaches the environment. A shield is synthesized from a temporal-logic specification and inserted around an agent, correcting or blocking unsafe actions at runtime (Alshiekh et al 2018). The primary object is the safety constraint.

Runtime agent enforcement asks whether agent behavior satisfies explicit runtime constraints, and what intervention follows when it does not. AgentSpec makes programmable rules primary, with triggers, predicates, and an enforcement mechanism that intervenes when a rule fires, for instance by requiring user confirmation before an action (Wang et al 2025). The primary object is the enforcement rule.

Proof of Execution asks whether an effectful execution followed its governing contract, and whether that execution can be independently validated and replayed. It makes the execution itself the object of verification, modeling an execution as a triple of a contract, an execution causal event stream, and a replay context. Its Prime Execution Model separates planning, enforcement, effect, and recordkeeping into distinct authority planes, and its validity predicate is a well-formedness condition together with five validator-checkable invariants, the semantic guarantees of authorization, path compliance, null effect on deny, history integrity, and replayability (Rhodes and Kang 2026). This is the closest neighbor on verification structure, and we do not minimize the overlap. Both separate proposal and planning from enforcement, effect, and recordkeeping, and both treat authorization at effectful execution as central. The distinction is one of object and research question. Proof of Execution defines validity for an execution object under a contract and stated deployment assumptions. This paper proposes a more abstract authority relation and asks which distinctions must be preserved and which transformations may be admitted for authority over a consequential action to remain invariant as the action moves toward execution, and it develops the relevance equivalence and non-expansion of section 6 around that question. Neither subsumes the other.

Classical authorization asks whether a subject may perform an operation on a resource under policy, and makes the subject, the resource, the action, and the policy evaluation primary (Sandhu et al 1996; NIST SP 800-162; OASIS XACML 3.0). Attested execution asks whether code ran in an environment possessing the identity and integrity properties on which trust depends, and makes the trustworthy environment primary (Sabt et al 2015).

A contemporaneous line of work, appearing through 2026, arrives independently at governing the act rather than the actor and decomposes it in different ways. Salfeld-Nebgen (2026) centers governance on consequential actions rather than agents: the agent keeps planning autonomy but holds no execution authority over high-risk actions, which are conditioned on independently attested preconditions, bound to a declared intent, evaluated by a deterministic policy, and recorded in tamper-evident logs. Proof-Carrying Agent Actions (Wang 2026a) makes a runtime-neutral action certificate primary, organizing control around checkpoints from pre-action admissibility through outcome closure. CAVA (Wang 2026b) canonicalizes heterogeneous runtime activity into canonical action objects for consistent approval and verification, which touches this paper’s representation pillar directly. Oswal and Cadeddu (2026) also propose five runtime primitives for governing autonomous agents, though theirs, discovery, identity, governance, attestation, and supply chain, are an operational decomposition rather than the STATE, PROPOSAL, AUTHORITY, ACTION, EVIDENCE of this paper.

A separate line of work asks what information must survive compression to preserve action adequacy rather than authority. Walsh formalizes support sufficiency as action-sufficient compression, whose coarsest exact form is the quotient of a support space by policy equivalence, two states merged when they require the same optimal action, with an approximate rate-regret version and a consequence-sensitive extension (Walsh 2026, action-sufficient compression; Walsh 2026, belief arbitration). This intersects the present paper at decision-relevant equivalence classes, since his quotient by policy equivalence and our quotient by authority-equivalence are the same construction, but the questions are different and we cite it as neighboring work, not as validation.

We therefore do not claim the phrase *govern the act, not the actor* as a differentiator, because it has become shared vocabulary. Given action-level governance, the question this paper develops is different from the primary questions addressed by these neighbors: which distinctions must survive transformations of authority-relevant state, and which may safely collapse. Our primary object is not an architecture, a certificate format, or a deployed governance system, but the authority relation itself, which requires no model of the worker or the substrate, together with the machine-checked equivalence, non-expansion, and relevance-soundness results, the transitive-closure hazard found during formalization, and the framing as a proposed general theory rather than a system.

10. Limits and open problems

The ontology remains proposed. We have not proved that five primitives are uniquely minimal or complete, only that the reduction retained five objects and that the formal development required no additional primitive.

The architecture remains proposed. The formal results of section 6 are consequences inside the model. They do not prove that R1, R2, and the relevance

principle universally characterize governed execution.

The formalization has scope. The false-abstention count is over an explicitly finite domain, the ordering of the negative verdicts is a modeling choice except where the unique authorizing top matters, and the authority relation abstracts whatever state a deployment declares authority-relevant.

The external evidence has grades. After a source-by-source audit, six mappings are direct instantiations, two are structural analogues, and the request-smuggling reading is our interpretation. Structural analogues are not counted as direct instantiations.

The implementation evidence is first-party and comes at three strengths we keep apart. The machine-checked bridge proves Figueroa quantization is a sound relevance criterion preserving the routing decision across every authored action, the node and fusion policies are implemented and tested, and the distributed fallback is observed on the scenarios and hardware walks exercised. Authored abstain and escalate are covered as actions by the proof, while the framework verdicts and the reuse of the governing shape across levels are tested rather than proved, with no composition theorem claimed. None of this independently validates the general theory, which a single originating corpus cannot.

The trust boundary remains, and making authority explicit does not remove it but makes it enumerable. A receipt establishes what the governed path accepted under its assumptions, and those assumptions can be named. Inside the trusted base sit the authority relation and how its policy was provisioned, the key material that signs and verifies decisions, and the time or version source the decision is evaluated against, so a substituted policy, a compromised key, or a lying clock each breaks governance from within. Outside sit the facts governance cannot establish internally, that a node is who it claims, that it runs the code it reports, that a sensor described the world truthfully, and that a physical effect occurred as represented unless separately witnessed. This is the provenance-is-not-truth limit of section 5 seen from the systems side. In the implementation record the boundary is stated per component, so an actuator's trusted base is the authority's key and the receiver's admission, and its own refusals are limited to signature, addressee, admission, and counter (ledger #157).

Semantic invariance does not imply resilience. Identical authorization semantics across many instances is the conformance property the theory requires, and it is also zero diversity. Because identical evaluators given the same input return the same outcome, a single decision-triggering input can drive every instance to the same refusal at once, a correlated failure. The theory preserves authorization semantics but does not claim identical semantics provide independent failure modes, and independent failure modes require diversity of function in addition to identical semantics.

11. Conclusion

We set out to govern the act rather than the actor, and to see whether one authorization abstraction could carry that across workers and substrates. The result is a small foundation, five primitives and two pillars joined by a relevance principle, with a machine-checked core that also exposes one design condition, a first-party implementation corpus, and an external countermodel search that found recurrence rather than the counterexample it sought. It is a proposed theory with a first-party corpus, not a proof of universality, and its value depends on whether it survives contact with systems and readers outside the work that produced it.

Three attacks would test it, one at each layer. At the level of ontology, exhibit a verifiably governed consequential execution for which the five-object model cannot represent the authority-bearing structure without an additional primitive. At the level of architecture, exhibit a recurring class of governed transformations for which authority equivalence and non-expansion fail to capture a distinction necessary to explain safe execution. At the level of the distinctions, exhibit a system satisfying this paper's definition of verifiable bounded permission that deliberately collapses one of the proposed distinctions while retaining the governance property that distinction is claimed to protect. Each targets a substantive claim of the theory rather than a definition, and a successful instance of any of the three would require revising the theory.

AI use disclosure

AI-assisted tools were used during the research and drafting of this paper, including research assistance, development support, and prose drafting and revision. These tools are not authors. A. Figueroa directed the work, determined the claims and interpretations presented, reviewed the manuscript and supporting artifacts, and accepts full responsibility for the content of the paper.

References

- Alshiekh, M., Bloem, R., Ehlers, R., Koenighofer, B., Niekum, S., and Topcu, U. Safe Reinforcement Learning via Shielding. Proceedings of the AAAI Conference on Artificial Intelligence (AAAI-18), pages 2669 to 2678, 2018. arXiv:1708.08611.
- Bell, D. E., and LaPadula, L. J. Secure Computer Systems: Mathematical Foundations. MITRE Technical Report 2547, 1973.
- Brucker, A. D., and Petritsch, H. Extending Access Control Models with Break-glass. Proceedings of the 14th ACM Symposium on Access Control Models and Technologies (SACMAT), pages 197 to 206, 2009. doi:10.1145/1542207.1542239.
- Codd, E. F. Extending the Database Relational Model to Capture More Meaning. ACM Transactions on Database Systems, 4(4), pages 397 to 434, 1979.

Kubernetes documentation. The Kubernetes Authors. Controlling Access to the Kubernetes API, and Admission Control in Kubernetes. <https://kubernetes.io/docs/concepts/security/controlling-access/> and <https://kubernetes.io/docs/reference/access-authn-authz/admission-controllers/>, accessed 30 September 2026.

Linhart, C., Klein, A., Heled, R., and Orrin, S. HTTP Request Smuggling. Watchfire white paper, 2005.

Loscocco, P., and Smalley, S. Integrating Flexible Support for Security Policies into the Linux Operating System. Proceedings of the FREENIX Track of the 2001 USENIX Annual Technical Conference, Boston, June 2001. USENIX Association.

NIST SP 800-162. Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R., and Scarfone, K. Guide to Attribute Based Access Control (ABAC) Definition and Considerations. National Institute of Standards and Technology, January 2014. doi:10.6028/NIST.SP.800-162.

OASIS XACML 3.0. Rissanen, E., editor. eXtensible Access Control Markup Language (XACML) Version 3.0. OASIS Standard, 22 January 2013. <http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.html>.

OpenID Connect Core 1.0. Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., and Mortimore, C. OpenID Connect Core 1.0 incorporating errata set 2. OpenID Foundation, 15 December 2023, first published as a final specification 25 February 2014. https://openid.net/specs/openid-connect-core-1_0.html.

Oswal, J., and Cadeddu, J. Five Primitives for Governing Autonomous AI Agents at Runtime, 2026. arXiv:2608.26696.

OWASP ASVS 5.0.0. OWASP Foundation. Application Security Verification Standard, version 5.0.0, requirement 1.1.1, May 2025. <https://owasp.org/www-project-application-security-verification-standard/>.

PortSwigger. HTTP request smuggling. Web Security Academy. <https://portswigger.net/web-security/request-smuggling>, accessed 30 September 2026.

PrismPath evidence ledger. Crystal Warden Labs. docs/research/supporting-evidence.md, version 2.14, rows #1 to #162, September 2026, each revision anchored with OpenTimestamps. <https://github.com/crystal-warden/prism-path>.

RFC 6749. Hardt, D., editor. The OAuth 2.0 Authorization Framework. Internet Engineering Task Force, October 2012. doi:10.17487/RFC6749.

RFC 8707. Campbell, B., Bradley, J., and Tschofenig, H. Resource Indicators for OAuth 2.0. Internet Engineering Task Force, February 2020. doi:10.17487/RFC8707.

Rhodes, J., and Kang, G. Proof of Execution: Runtime Verification for Governed AI Agent Actions, 2026. arXiv:2607.05397.

- Richardson, C. *Microservices Patterns*, with examples in Java. Manning, 2018. ISBN 9781617294549.
- Sabt, M., Achemlal, M., and Bouabdallah, A. Trusted Execution Environment: What It is, and What It is Not. 2015 IEEE Trustcom/BigDataSE/ISPA, pages 57 to 64, 2015. doi:10.1109/Trustcom.2015.357.
- Salfeld-Nebgen, J. Governing Actions, Not Agents: Institutional Attestation as a Governance Model for Autonomous AI Systems, 2026. arXiv:2606.26298.
- Saltzer, J. H., and Schroeder, M. D. The Protection of Information in Computer Systems. *Proceedings of the IEEE*, 63(9), pages 1278 to 1308, 1975.
- Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. Role-Based Access Control Models. *IEEE Computer*, 29(2), pages 38 to 47, 1996.
- SLSA specification. SLSA community, an OpenSSF project. Supply-chain Levels for Software Artifacts, specification version 1.2, 24 November 2025. <https://slsa.dev/spec/v1.2/>.
- Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. in-toto: Providing farm-to-table guarantees for bits and bytes. *Proceedings of the 28th USENIX Security Symposium*, pages 1393 to 1410, 2019.
- Unicode UTS 39. Hadley, J., and Pournader, R., editors. Unicode Security Mechanisms, Unicode Technical Standard #39, version 18.0.0. Unicode Consortium, 27 August 2026. <https://www.unicode.org/reports/tr39/>.
- Waldron, W. Understanding the Dual-Write Problem and Its Solutions. *Confluent blog*, 29 May 2024. <https://www.confluent.io/blog/dual-write-problem/>.
- Walsh, M. Support Sufficiency as Consequence-Sensitive Compression in Belief Arbitration, 2026. arXiv:2604.16434.
- Walsh, M. Support sufficiency as action-sufficient compression: a single-cycle rate-regret formulation, 2026. arXiv:2606.09858.
- Wang, H., Poskitt, C. M., and Sun, J. AgentSpec: Customizable Runtime Enforcement for Safe and Reliable LLM Agents, 2025. arXiv:2503.18666 (ICSE 2026).
- Wang, Z. Proof-Carrying Agent Actions: Model-Agnostic Runtime Governance for Heterogeneous Agent Systems, 2026a. arXiv:2606.04104.
- Wang, Z. CAVA: Canonical Action Verification and Attestation for Runtime Governance of Agentic AI Systems, 2026b. arXiv:2607.13716.

Appendix A. The dictionary reduction

The vocabulary was reduced, not invented. We began with the source implementation's working dictionary, `docs/Dictionary.md` in the PrismPath repository at commit d4b13bf, of about one hundred and twenty terms and classified each

as a candidate primitive, a concept derived from primitives, a general mechanism realizing a concept, an implementation detail specific to the system, or a record of what happened. The first-pass dictionary classified authority, outcome, reading, and evidence as primitive terms. Those labels informed the theoretical reduction but were not carried over as a one-to-one ontology. In particular the theory distinguishes the candidate transition, the ruling on that candidate, and the consequential transition whose occurrence is governed. The original vocabulary did not name that structure cleanly, and the resulting distinction between PROPOSAL, derived DECISION, and ACTION became part of the proposed model. The five-primitive set is therefore the result of reduction and clarification, not simply the original four dictionary primitives with one name added. The full per-term classification is in the companion artifact `dictionary_reduction_firstpass.md`.

The reduction is visible in the originating implementation. A signed policy table on the mesh, a signed policy pack on a camera, and a receiver's held normal have different implementation identities, but they reuse a small set of mechanisms, a signature by the fleet authority verified against a key baked into the device, a monotonic version floor that refuses rollback and replay, and a freshness bound after which an input is explicitly stale (ledger #102, #125, #149, #150). At the theoretical level they contribute authority-relevant state, authenticity distinct from authority, temporal validity, or evidence, rather than requiring new primitives, so they classify as mechanism and state rather than as additions to the primitive set. The same implementation carries freshness, a replay window, a policy version floor, a held normal, and quorum, all of which affect authorization and all of which the model represents as components of the authority-relevant state on which A depends, rather than as a fixed schema (ledger #100, #125, #126, #149, #150). Accordingly A is written $A(\sigma, a)$ with σ open, and time enters as a component of state rather than as a separate primitive.

Appendix B. The formal development

All definitions and theorems below are in `lean-governance/Governance.lean`, which imports neither a mathematics library nor the implementation corpus, except the final five which are in `lean-governance/FQBridge.lean` and import the corpus. The check recipe is `cd lean-governance && ./check.sh`, which checks the core alone and then the bridge, and the toolchain is `leanprover/lean4:v4.33.1`.

Definitions. `Permissive` (an outcome preorder, no more permissive than); `Authority` (state and action to outcome); `simA` (the state equivalence induced by authority); `Domain`; `Admissible` (a transform preserving the committed outcome); `NonExpanding` (a transform never more permissive); `Induces` (a transform respects a criterion); `Sound` and `SoundConservative` (criterion soundness, exact and directional); `TC` (transitive closure of a criterion); `Complete` (the converse of `Sound`); `AnalysisSound`, `AnalysisComplete`, `FalseAbstain`, `HasFloor`

(the analysis and its floor); Verdict (the five valued codomain); faCount (the false-abstention count over a finite domain); RelevanceAnalysis, Exact, Safe (the relevance layer). fqR and fqA (the FQ relevance criterion and its per action routing authority, in FQBridge.lean).

Theorems. `simA_equiv`, the state equivalence is reflexive, symmetric, transitive. `admissible_preserves`, an admissible transform lands a state in its own authority class. `admissible_nonexpanding`, admissibility implies non-expansion. `nonexp_not_admissible`, non-expansion is strictly weaker (a witness). `admissible_comp` and `nonexpanding_comp`, both properties are closed under composition. `exact_safe` and `safe_not_exact`, the relevance restatements. `sound_induces_admissible`, a sound criterion with an induced transform is admissible. `soundcons_induces_nonexpanding`, the conservative tier gives non-expansion. `sound_tc`, exact soundness is preserved by transitive closure. `weak_criterion_not_closed`, an overlap criterion is not (a witness). `sound_complete_char`, sound and complete is exactly authority-equality. `complete_no_false_abstain`, analysis completeness rules out false abstention. `sound_allows_false_abstain`, soundness alone does not (a witness). `sound_induces_complete_analysis`, a sound criterion with a total induced transform is complete. `verdict_authorize_top`, `verdict_refuse_bot`, `verdict_rubberstamp_unsafe`, `verdict_escalate_safe`, the five valued codomain and its top. `faCount_le_length`, the false-abstention count is bounded by the domain size, so the normalized rate is at most one. `faCount_eq_zero_of_no_false_abstain` and `faCount_eq_zero_of_complete`, a zero count. `faCount_mono`, a more committal analysis has a no larger count. `faCount_strictly_decreases`, a concrete strict drop. `sound_total_faCount_zero`, a sound criterion that places every state has a zero count. `fq_route_eq`, well typed readings with equal quantization produce the same routing result, from `decision_preservation`. `fq_multi_action_equiv`, those readings carry the same authority for every action in the policy's domain, the multi-action equivalence. `fq_sound`, Figueroa quantization is a sound relevance criterion in the sense of section 6.3, the concrete realization of the relevance principle. `fq_admissible`, a transformation preserving quantization and well typedness on a domain is admissible for the routing authority. `fq_nonexpanding`, non-expansion follows through the general safety arrow. The last five are in FQBridge.lean and are discharged from `decision_preservation` over the policy's actual action domain.

Appendix C. External mappings, with grades

A1 through A8 are the eight distinctions of section 5. A1, A2, A3, A5, A7, and A8 are the six separations the two roots generate, A4 is the boundary condition of the relevance principle, and A6 is the semantic limit of the evidence primitive. All eight are graded on the same terms, since the external search asks of each whether an independent system keeps it.

Direct instantiation: A1 capability vs authority, mandatory over discretionary access control (Bell and LaPadula 1973; Loscocco and Smalley 2001). A2 pro-

positional vs authorization, the XACML policy enforcement and decision points (OASIS XACML 3.0). A3 authenticity vs authority, authentication is not authorization (RFC 6749; OpenID Connect Core 1.0). A4 absence vs explicit state, fail-safe defaults (Saltzer and Schroeder 1975; Codd 1979). A5 representation vs semantics, canonicalize before deciding (Unicode UTS 39; OWASP ASVS 5.0.0). A6 provenance vs truth, provenance not correctness (SLSA specification; Torres-Arias et al 2019).

Structural analogue: A7 sequencing vs authority, admission separate from scheduling (Kubernetes documentation), gap stated. A8 execution vs evidence, the dual-write problem (Richardson 2018; Waldron 2024), where the external record is weaker than the theory's evidence.

Theory interpretation: HTTP request smuggling read through A5 (Linhart et al 2005; PortSwigger), the authors' reading rather than external validation.

Derivation before lookup, applying R1 to urgency, yielded an urgency-is-not-authority separation written down before the external search, which then found break-glass access controls as an existing realization (Brucker and Petritsch 2009). Our working notes record the urgency candidate before the search, without an independent timestamp, and it is not an independent prospective prediction.

Supplementary artifacts

The companion artifacts ship at stable relative paths, mapped on release to a public archive with a persistent identifier such as a DOI.

- `supplementary/CITATION_AUDIT.md`, the source-by-source citation audit behind sections 8 and 9.
- `supplementary/dictionary_reduction_firstpass.md`, the per-term classification behind appendix A.
- `supplementary/lean-governance/`, the machine-checked formal development of section 6 and appendix B, with the check script `check.sh` and pinned toolchain.